TECHNICAL DOCUMENT

Kafka Event Catalog

docs/KAFKA_EVENT_CATALOG.md

Topics · Events · Producers · Consumers

PLATFORM	STACK	GENERATED
EduTech v1.0	Java 17 · Spring Boot 3.3.5	March 07, 2026

Kafka Event Catalog — EduTech AI Platform

Last updated: 2026-03-07
Kafka version: 3.6 (KRaft mode — no Zookeeper)
Serialization: JSON via Spring Kafka `JsonSerializer` / `JsonDeserializer` (Jackson)
Type headers: disabled (`spring.json.add.type.headers: false`)

Table of Contents

1. Overview
2. Topic Registry
3. Publishing Pattern
4. Event Catalog
5. auth-svc
6. center-svc
7. parent-svc
8. assess-svc
9. psych-svc
10. ai-gateway-svc
11. Consumer Notes
12. Audit Immutable Topic

Overview

All domain events in the EduTech AI Platform are Java `record` types — immutable by construction. Every event carries a unique `eventId` (UUID) and an `occurredAt` (Instant, UTC) field for ordering and deduplication, with the exception of psych-svc and ai-gateway-svc events which are early-stage and omit the timestamp field pending schema stabilization.

Key invariants:

- Publishing is **best-effort**: a Kafka publish failure is logged but never causes the enclosing database transaction to roll back. The primary persistence operation always takes precedence.
- Every significant domain event is **dual-published**: once to the service-specific topic and once to `audit-immutable`.
- Topic names are **never hardcoded** in application code. All topic names are injected at runtime via environment variables (see `kafka.topics.*` in each service's `application.yml`).
- Message key is the `eventId` (string) to ensure partition locality per event.

Topic Registry

ENVIRONMENT VARIABLE	DEFAULT / EXAMPLE NAME	RETENTION	PRODUCERS	CONSUMERS
`KAFKA_TOPIC_AUTH_EVENTS`	`auth-events`	7 days	auth-svc	notification-svc (planned), analytics-svc (planned)
`KAFKA_TOPIC_CENTER_EVENTS`	`center-events`	7 days	center-svc	parent-svc, assess-svc, analytics-svc (planned)
`KAFKA_TOPIC_PARENT_EVENTS`	`parent-events`	7 days	parent-svc	notification-svc (planned), analytics-svc (planned)
`KAFKA_TOPIC_ASSESS_EVENTS`	`assess-events`	7 days	assess-svc	psych-svc, analytics-svc (planned)
`KAFKA_TOPIC_PSYCH_EVENTS`	`psych-events`	7 days	psych-svc	analytics-svc (planned)
`KAFKA_TOPIC_AI_GATEWAY_EVENTS`	`ai-gateway-events`	7 days	ai-gateway-svc	analytics-svc (planned)

ENVIRONMENT VARIABLE	DEFAULT / EXAMPLE NAME	RETENTION	PRODUCERS	CONSUMERS
`KAFKA_TOPIC_AUDIT_IMMUTABLE`	`audit-immutable`	**Forever (no expiry)**	auth-svc, center-svc, parent-svc, assess-svc, psych-svc, ai-gateway-svc	compliance-svc (planned)
`KAFKA_TOPIC_NOTIFICATION_SEND`	`notification-send`	3 days	auth-svc	notification-svc (planned)

> **Note on `center-events`: parent-svc and assess-svc declare `center-events` in their topic config, indicating they consume or cross-publish to that topic. Confirm consumer group registrations when activating these consumers.**

Publishing Pattern

All Kafka adapters follow the same best-effort, non-blocking pattern using Spring Kafka's `KafkaTemplate`:

```
// Best-effort — never throws, never rolls back the DB transaction
CompletableFuture<SendResult<String, Object>> future =
    kafkaTemplate.send(topic, eventId.toString(), event);

future.whenComplete((result, ex) -> {
    if (ex != null) {
        log.error("Failed to publish event type={} topic={}",
            event.getClass().getSimpleName(), topic, ex);
    } else {
        log.debug("Event published type={} partition={} offset={}",
            event.getClass().getSimpleName(),
            result.getRecordMetadata().partition(),
            result.getRecordMetadata().offset());
    }
});
```

Port / Adapter pattern: Each service exposes a domain port (e.g., `AuditEventPublisher`, `CenterEventPublisher`) and wires the Kafka adapter as the production implementation. This keeps domain logic free of Kafka dependencies.

Event Catalog

auth-svc

Package root: `com.edutech.auth.domain.event`
Kafka adapter: `com.edutech.auth.infrastructure.kafka.AuditEventKafkaAdapter`

`UserRegisteredEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`userId`	`UUID`	Newly created user's identifier
`email`	`String`	User's email address
`role`	`Role`	Assigned role enum (e.g., `CENTER_ADMIN`, `TEACHER`, `PARENT`, `STUDENT`)
`centerId`	`UUID`	Associated center; may be `null` for `SUPER_ADMIN`
`occurredAt`	`Instant`	UTC timestamp of account creation

Trigger: A new user account is successfully persisted to the database (`POST /api/v1/auth/register`).

Topic: `KAFKA_TOPIC_AUTH_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "userId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "email": "admin@brightacademy.com",
  "role": "CENTER_ADMIN",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "occurredAt": "2026-03-07T08:30:00.000Z"
}
```

`UserAuthenticatedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`userId`	`UUID`	Authenticated user's identifier; `null` on failed attempts
`email`	`String`	Email used in the login attempt
`ipAddress`	`String`	Client IP address

FIELD	TYPE	DESCRIPTION
`userAgent`	`String`	Raw `User-Agent` header
`success`	`boolean`	`true` for successful logins, `false` for failures
`failureReason`	`String`	Human-readable failure description; `null` on success
`occurredAt`	`Instant`	UTC timestamp of the attempt

Trigger: Every login attempt, regardless of outcome (`POST /api/v1/auth/login`). Both successful and failed attempts are recorded to support anomaly detection.

Topic: `KAFKA_TOPIC_AUTH_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON (successful login):

```
{
  "eventId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "userId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "email": "teacher@brightacademy.com",
  "ipAddress": "203.0.113.45",
  "userAgent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36",
  "success": true,
  "failureReason": null,
  "occurredAt": "2026-03-07T09:15:22.341Z"
}
```

Sample JSON (failed login):

```
{
  "eventId": "b5dc4580-93ec-4c1e-ae22-fd91b0d22c1a",
  "userId": null,
  "email": "unknown@example.com",
  "ipAddress": "198.51.100.77",
  "userAgent": "python-requests/2.31.0",
  "success": false,
  "failureReason": "INVALID_CREDENTIALS",
  "occurredAt": "2026-03-07T09:15:30.100Z"
}
```

`TokenRefreshedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`userId`	`UUID`	Owner of the refresh token
`oldTokenId`	`String`	JTI of the consumed (now-invalidated) refresh token
`newTokenId`	`String`	JTI of the newly issued refresh token
`occurredAt`	`Instant`	UTC timestamp of the rotation

Trigger: A valid refresh token is consumed and a new access + refresh token pair is issued (`POST /api/v1/auth/refresh`). Enables detection of refresh token reuse attacks by correlating `oldTokenId` across events.

Topic: `KAFKA_TOPIC_AUTH_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "d290f1ee-6c54-4b01-90e6-d701748f0851",
  "userId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "oldTokenId": "rt-prev-9f8e7d6c-5b4a-3210-fedc-ba9876543210",
  "newTokenId": "rt-next-1a2b3c4d-5e6f-7890-abcd-ef0123456789",
  "occurredAt": "2026-03-07T09:30:00.000Z"
}
```

`UserLogoutEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`userId`	`UUID`	User performing the logout
`tokenId`	`String`	JTI of the refresh token being invalidated
`allSessions`	`boolean`	`true` if logout-all-devices was requested
`occurredAt`	`Instant`	UTC timestamp of logout

Trigger: User logout (`POST /api/v1/auth/logout`). When `allSessions = true` , all entries in the Redis set `rt:user:{userId}` are deleted; when `false` , only the specific `rt:{tokenId}` key is deleted.

Topic: `KAFKA_TOPIC_AUTH_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "550e8400-e29b-41d4-a716-446655440000",
  "userId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "tokenId": "rt-next-1a2b3c4d-5e6f-7890-abcd-ef0123456789",
  "allSessions": false,
  "occurredAt": "2026-03-07T17:00:00.000Z"
}
```

`OtpRequestedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`email`	`String`	Recipient email address
`purpose`	`String`	OTP purpose: `EMAIL_VERIFICATION`, `PASSWORD_RESET`, or `LOGIN_2FA`
`channel`	`String`	Delivery channel: `EMAIL` or `SMS`
`occurredAt`	`Instant`	UTC timestamp of OTP generation

Trigger: An OTP is generated and stored in Redis (`POST /api/v1/otp/send`). This event triggers the notification pipeline to deliver the OTP via the specified channel. The OTP value itself is **never** included in the event.

Topic: `KAFKA_TOPIC_AUTH_EVENTS` and `KAFKA_TOPIC_NOTIFICATION_SEND`

Sample JSON:

```
{
  "eventId": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "email": "parent@example.com",
  "purpose": "EMAIL_VERIFICATION",
  "channel": "EMAIL",
  "occurredAt": "2026-03-07T10:00:00.000Z"
}
```

---

center-svc

Package root: `com.edutech.center.domain.event`
Kafka adapter: `com.edutech.center.infrastructure.kafka.CenterEventKafkaAdapter`

`BatchCreatedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`batchId`	`UUID`	Newly created batch identifier
`centerId`	`UUID`	Center that owns this batch
`batchName`	`String`	Display name of the batch
`subject`	`String`	Subject taught in this batch
`teacherId`	`UUID`	Initially assigned teacher
`occurredAt`	`Instant`	UTC timestamp of batch creation

Trigger: A CENTER_ADMIN creates a new batch within their center.

Topic: `KAFKA_TOPIC_CENTER_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "1b9d6bcd-bbfd-4b2d-9b5d-ab8dfbbd4bed",
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "batchName": "Grade 10 - Mathematics (Morning)",
  "subject": "MATHEMATICS",
  "teacherId": "t1e2a3c4-h5e6-7890-abcd-ef1234567890",
  "occurredAt": "2026-03-07T08:00:00.000Z"
}
```

`BatchStatusChangedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`batchId`	`UUID`	Affected batch
`centerId`	`UUID`	Owning center
`previousStatus`	`BatchStatus`	Status before the transition
`newStatus`	`BatchStatus`	Status after the transition
`occurredAt`	`Instant`	UTC timestamp of the status change

Trigger: A batch lifecycle state transition (e.g., `DRAFT` → `ACTIVE`, `ACTIVE` → `COMPLETED`, `ACTIVE` → `CANCELLED`).

Topic: `KAFKA_TOPIC_CENTER_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "previousStatus": "DRAFT",
  "newStatus": "ACTIVE",
  "occurredAt": "2026-03-07T08:30:00.000Z"
}
```

`TeacherAssignedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`teacherId`	`UUID`	Teacher profile identifier (center-svc domain entity)
`centerId`	`UUID`	Center to which the teacher is assigned
`userId`	`UUID`	Auth-svc user identifier for this teacher
`email`	`String`	Teacher's email address
`occurredAt`	`Instant`	UTC timestamp of assignment

Trigger: A user with role `TEACHER` is formally assigned to a center by a CENTER_ADMIN or SUPER_ADMIN.

Topic: `KAFKA_TOPIC_CENTER_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "6ba7b811-9dad-11d1-80b4-00c04fd430c8",
  "teacherId": "t1e2a3c4-h5e6-7890-abcd-ef1234567890",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "userId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "email": "teacher@brightacademy.com",
  "occurredAt": "2026-03-07T09:00:00.000Z"
}
```

`ScheduleChangedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`scheduleId`	`UUID`	The modified schedule entry
`batchId`	`UUID`	Batch to which this schedule belongs
`centerId`	`UUID`	Owning center
`changeType`	`String`	Type of change (e.g., `RESCHEDULED`, `CANCELLED`, `ADDED`)
`occurredAt`	`Instant`	UTC timestamp of the schedule change

Trigger: A schedule entry for a batch is created, modified, or cancelled (e.g., a class is moved to a different time slot).

Topic: `KAFKA_TOPIC_CENTER_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "9ecad4b5-1a2b-3c4d-5e6f-7890abcdef01",
  "scheduleId": "sch-001-2026-03-10",
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "changeType": "RESCHEDULED",
  "occurredAt": "2026-03-07T11:00:00.000Z"
}
```

`ContentUploadedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`contentId`	`UUID`	Newly created content record identifier
`centerId`	`UUID`	Center that owns this content
`batchId`	`UUID`	Batch this content is assigned to
`title`	`String`	Display title of the content
`type`	`ContentType`	Content type enum (e.g., `VIDEO`, `PDF`, `QUIZ`, `ASSIGNMENT`)

FIELD	TYPE	DESCRIPTION
`uploadedByUserId`	`UUID`	Auth-svc user ID of the uploader
`occurredAt`	`Instant`	UTC timestamp of upload completion

Trigger: A teacher or CENTER_ADMIN successfully uploads or links content to a batch.

Topic: `KAFKA_TOPIC_CENTER_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "f0e1d2c3-b4a5-6789-0abc-def012345678",
  "contentId": "cont-aabb-ccdd-eeff-001122334455",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "title": "Chapter 5 - Quadratic Equations (Lecture Recording)",
  "type": "VIDEO",
  "uploadedByUserId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "occurredAt": "2026-03-07T14:20:00.000Z"
}
```

---

parent-svc

Package root: `com.edutech.parent.domain.event`

`StudentLinkedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`linkId`	`UUID`	Parent-student link record identifier
`parentId`	`UUID`	Parent profile identifier
`studentId`	`UUID`	Student user identifier (auth-svc)
`centerId`	`UUID`	Center where the student is enrolled
`occurredAt`	`Instant`	UTC timestamp of link creation

Trigger: A parent successfully links their account to a student account at a center, establishing the guardian relationship.

Topic: `KAFKA_TOPIC_PARENT_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "12345678-1234-5678-1234-567812345678",
  "linkId": "lnk-0001-aaaa-bbbb-cccc-000000000001",
  "parentId": "par-0001-dddd-eeee-ffff-000000000002",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "occurredAt": "2026-03-07T10:05:00.000Z"
}
```

`LinkRevokedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`linkId`	`UUID`	Parent-student link record identifier being revoked
`parentId`	`UUID`	Parent profile identifier
`studentId`	`UUID`	Student user identifier
`occurredAt`	`Instant`	UTC timestamp of revocation

Trigger: A parent-student guardian link is explicitly revoked by a CENTER_ADMIN or SUPER_ADMIN.

Topic: `KAFKA_TOPIC_PARENT_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "abcdef12-3456-7890-abcd-ef1234567890",
  "linkId": "lnk-0001-aaaa-bbbb-cccc-000000000001",
  "parentId": "par-0001-dddd-eeee-ffff-000000000002",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "occurredAt": "2026-03-07T15:00:00.000Z"
}
```

`FeePaymentRecordedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)

FIELD	TYPE	DESCRIPTION
`paymentId`	`UUID`	Payment record identifier
`parentId`	`UUID`	Parent making the payment
`studentId`	`UUID`	Student for whom fees are paid
`centerId`	`UUID`	Center receiving the fee
`batchId`	`UUID`	Specific batch the fee is for
`amountPaid`	`BigDecimal`	Payment amount (exact decimal, no floating-point)
`currency`	`String`	ISO 4217 currency code (e.g., `INR`, `USD`)
`occurredAt`	`Instant`	UTC timestamp of payment recording

Trigger: A fee payment is recorded for a student's enrollment in a batch (payment gateway webhook processed or manual entry by CENTER_ADMIN).

Topic: `KAFKA_TOPIC_PARENT_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "fee00001-1234-5678-abcd-ef0000000001",
  "paymentId": "pay-aaaa-bbbb-cccc-dddd-eeeeeeeeeeee",
  "parentId": "par-0001-dddd-eeee-ffff-000000000002",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "amountPaid": "12500.00",
  "currency": "INR",
  "occurredAt": "2026-03-07T11:30:00.000Z"
}
```

---

assess-svc

Package root: `com.edutech.assess.domain.event`

`ExamPublishedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)

FIELD	TYPE	DESCRIPTION
`examId`	`UUID`	Published exam identifier
`batchId`	`UUID`	Batch for which this exam is published
`centerId`	`UUID`	Owning center
`title`	`String`	Exam display title
`totalMarks`	`double`	Maximum achievable marks
`occurredAt`	`Instant`	UTC timestamp when the exam was published

Trigger: A teacher or CENTER_ADMIN transitions an exam from draft to published state, making it visible and accessible to enrolled students.

Topic: `KAFKA_TOPIC_ASSESS_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "exam0001-aaaa-bbbb-cccc-dddddddddddd",
  "examId": "e1f2g3h4-i5j6-7890-abcd-ef1234567890",
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "title": "Unit Test 2 - Quadratic Equations",
  "totalMarks": 100.0,
  "occurredAt": "2026-03-07T09:00:00.000Z"
}
```

`ExamSubmittedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`submissionId`	`UUID`	Submission record identifier
`examId`	`UUID`	The exam being submitted
`studentId`	`UUID`	Submitting student
`scoredMarks`	`double`	Marks scored by the student
`totalMarks`	`double`	Maximum possible marks
`occurredAt`	`Instant`	UTC timestamp of submission

Trigger: A student submits their exam answers (either manual submission or auto-submit on timer expiry). The system calculates `scoredMarks` before publishing this event.

Topic: `KAFKA_TOPIC_ASSESS_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "sub00001-1111-2222-3333-444444444444",
  "submissionId": "s9t8u7v6-w5x4-y3z2-a1b0-c9d8e7f65432",
  "examId": "e1f2g3h4-i5j6-7890-abcd-ef1234567890",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "scoredMarks": 78.5,
  "totalMarks": 100.0,
  "occurredAt": "2026-03-07T11:45:30.000Z"
}
```

`GradeIssuedEvent`

FIELD	TYPE	DESCRIPTION
`eventId`	`UUID`	Unique event identifier (auto-generated)
`gradeId`	`UUID`	Grade record identifier
`submissionId`	`UUID`	Associated submission
`examId`	`UUID`	Graded exam
`studentId`	`UUID`	Student receiving the grade
`batchId`	`UUID`	Batch context
`centerId`	`UUID`	Center context
`percentage`	`double`	Percentage score (0.0–100.0)
`passed`	`boolean`	Whether the student met the passing threshold
`occurredAt`	`Instant`	UTC timestamp of grade issuance

Trigger: Grading completes (auto-graded or manually graded by teacher) and a formal grade record is persisted. psych-svc consumes this event to update psychometric profiles.

Topic: `KAFKA_TOPIC_ASSESS_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "eventId": "grd00001-5555-6666-7777-888888888888",
  "gradeId": "g0h1i2j3-k4l5-6789-mnop-qrstuvwxyz01",
  "submissionId": "s9t8u7v6-w5x4-y3z2-a1b0-c9d8e7f65432",
  "examId": "e1f2g3h4-i5j6-7890-abcd-ef1234567890",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "percentage": 78.5,
  "passed": true,
  "occurredAt": "2026-03-07T12:00:00.000Z"
}
```

---

psych-svc

Package root: `com.edutech.psych.domain.event`

> **Note:** psych-svc events are currently lightweight value objects without `eventId` or `occurredAt` fields. This is an early-stage schema — these fields are expected to be added in a future iteration to align with the platform-wide event contract standard.

`PsychProfileCreatedEvent`

FIELD	TYPE	DESCRIPTION
`profileId`	`UUID`	Newly created psychometric profile identifier
`studentId`	`UUID`	Student to whom this profile belongs
`centerId`	`UUID`	Center context
`batchId`	`UUID`	Initial batch context at profile creation time

Trigger: A psychometric profile is created for a student upon enrollment or explicit initiation by a counsellor.

Topic: `KAFKA_TOPIC_PSYCH_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "profileId": "prf-0001-aaaa-bbbb-cccc-000000000010",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
```

```
  "batchId": "ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456"
}
```

`SessionCompletedEvent`

FIELD	TYPE	DESCRIPTION
`sessionId`	`UUID`	Completed psychometric session identifier
`profileId`	`UUID`	Profile updated by this session
`studentId`	`UUID`	Student who completed the session
`centerId`	`UUID`	Center context

Trigger: A student completes a psychometric assessment session (questionnaire, behavioral test, or counsellor-recorded session).

Topic: `KAFKA_TOPIC_PSYCH_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "sessionId": "sess-1234-abcd-efgh-ijkl-000000000020",
  "profileId": "prf-0001-aaaa-bbbb-cccc-000000000010",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001"
}
```

`CareerMappingGeneratedEvent`

FIELD	TYPE	DESCRIPTION
`mappingId`	`UUID`	Generated career mapping record identifier
`profileId`	`UUID`	Source psychometric profile
`studentId`	`UUID`	Student for whom the mapping was generated
`centerId`	`UUID`	Center context
`topCareers`	`String`	Comma-separated or JSON-serialized list of top career recommendations

Trigger: The AI career prediction pipeline (Python FastAPI sidecar) completes analysis of a student's psychometric profile and produces ranked career recommendations.

Topic: `KAFKA_TOPIC_PSYCH_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "mappingId": "map-5678-efgh-ijkl-mnop-000000000030",
  "profileId": "prf-0001-aaaa-bbbb-cccc-000000000010",
  "studentId": "stu-0001-1111-2222-3333-000000000003",
  "centerId": "c0ffee00-dead-beef-cafe-000000000001",
  "topCareers": "Software Engineer,Data Scientist,UX Designer,Product Manager"
}
```

---

ai-gateway-svc

Package root: `com.edutech.aigateway.domain.event`

> **Note:** ai-gateway-svc events use `String` identifiers rather than `UUID` to accommodate non-UUID correlation IDs from upstream LLM providers. Neither `eventId` nor `occurredAt` fields are present in the current schema.

`AiRequestRoutedEvent`

FIELD	TYPE	DESCRIPTION
`requestId`	`String`	Unique request correlation identifier
`requesterId`	`String`	Identifier of the calling service or user (used for rate limiting)
`modelType`	`ModelType`	Target model type enum (e.g., `CHAT`, `EMBEDDING`, `CAREER_PREDICTION`)
`provider`	`LlmProvider`	LLM provider used (e.g., `ANTHROPIC`, `OPENAI`, `OLLAMA`)
`latencyMs`	`long`	End-to-end latency in milliseconds from request receipt to response

Trigger: An AI request is successfully routed to an LLM provider and a response is received. Published after successful completion to enable cost tracking and provider performance analytics.

Topic: `KAFKA_TOPIC_AI_GATEWAY_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "requestId": "req-abc123-def456-ghi789",
  "requesterId": "center-svc:a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "modelType": "CHAT",
```

```
  "provider": "ANTHROPIC",
  "latencyMs": 1423
}
```

`AiRequestFailedEvent`

FIELD	TYPE	DESCRIPTION
`requestId`	`String`	Unique request correlation identifier
`requesterId`	`String`	Identifier of the calling service or user
`modelType`	`ModelType`	Target model type that failed
`errorMessage`	`String`	Human-readable description of the failure

Trigger: An AI request fails after all Resilience4j retry/circuit-breaker attempts are exhausted. Published to enable alerting on provider outages and error pattern analysis.

Topic: `KAFKA_TOPIC_AI_GATEWAY_EVENTS` and `KAFKA_TOPIC_AUDIT_IMMUTABLE`

Sample JSON:

```
{
  "requestId": "req-xyz789-uvw456-rst123",
  "requesterId": "assess-svc:ba7cb952-1f37-4a8e-b1a2-3f9c0d12e456",
  "modelType": "EMBEDDING",
  "errorMessage": "Anthropic API returned HTTP 529: Overloaded. Circuit breaker OPEN after 5 failu
res."
}
```

Consumer Notes

As of this writing, all Kafka topics in the system have **producers only** — there are no in-repository consumer implementations. The event-driven architecture is designed for extensibility:

PLANNED CONSUMER SERVICE	TOPICS IT WILL CONSUME	PURPOSE
`notification-svc`	`auth-events`, `notification-send`, `assess-events`, `center-events`	Email, SMS, and push notification delivery
`analytics-svc`	All service topics	Business intelligence, usage dashboards, anomaly detection

PLANNED CONSUMER SERVICE	TOPICS IT WILL CONSUME	PURPOSE
`compliance-svc`	`audit-immutable`	Regulatory reporting, data retention policy enforcement

Consumer group IDs are externalized via environment variable per service (e.g., `AUTH_SVC_KAFKA_CONSUMER_GROUP`) and configured with `auto-offset-reset: earliest` to ensure no events are missed on new consumer group startup.

Audit Immutable Topic

The `audit-immutable` topic (`KAFKA_TOPIC_AUDIT_IMMUTABLE`) is the platform's central compliance log.

Properties:

- **Retention:** No expiry (retain forever / subject to storage-based compaction policy)
- **Access:** Append-only from the application layer — no update or delete operations
- **Content:** Receives a copy of every event from every service that carries compliance significance
- **Purpose:** Provides a tamper-evident chronological record of all authentication, enrollment, financial, and assessment events
- **Events routed here:** `UserRegisteredEvent`, `UserAuthenticatedEvent`, `TokenRefreshedEvent`, `UserLogoutEvent`, `OtpRequestedEvent`, `BatchCreatedEvent`, `BatchStatusChangedEvent`, `TeacherAssignedEvent`, `ScheduleChangedEvent`, `ContentUploadedEvent`, `StudentLinkedEvent`, `LinkRevokedEvent`, `FeePaymentRecordedEvent`, `ExamPublishedEvent`, `ExamSubmittedEvent`, `GradeIssuedEvent`, `PsychProfileCreatedEvent`, `SessionCompletedEvent`, `CareerMappingGeneratedEvent`, `AiRequestRoutedEvent`, `AiRequestFailedEvent`

The `AuditEventKafkaAdapter` in auth-svc is the reference implementation. Each service implements an equivalent adapter that publishes to both its own topic and `audit-immutable`:

```
// From AuditEventKafkaAdapter — reference implementation
@Override
public void publish(Object event) {
    String topic = topicProperties.auditImmutable();
    CompletableFuture<SendResult<String, Object>> future =
        kafkaTemplate.send(topic, event);

    future.whenComplete((result, ex) -> {
        if (ex != null) {
            log.error("Failed to publish audit event type={} topic={}",
                event.getClass().getSimpleName(), topic, ex);
        } else {
            log.debug("Audit event published type={} partition={} offset={}",
                event.getClass().getSimpleName(),
                result.getRecordMetadata().partition(),
                result.getRecordMetadata().offset());
```

```
        }
    });
}
```